

# GUIA RESUMEN — Clean Architecture para Flutter

Resumen integral para dominar Clean Architecture en ~25 minutos, sin volver a leer el modulo completo.

## 0 Flujo de aprendizaje general

4 PARTES · ~25 MIN

|                                                      |                                                          |                                                          |                                                  |
|------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------|
| <div>1</div> <div>Fundamentos</div> <div>8 min</div> | <div>2</div> <div>Implementacion</div> <div>10 min</div> | <div>3</div> <div>Infraestructura</div> <div>4 min</div> | <div>4</div> <div>Madurez</div> <div>3 min</div> |
|------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------|

Regla de oro: "Las flechas siempre apuntan hacia adentro (hacia Domain). Domain nunca sabe de las capas externas."

| Parte              | Que aprendes                                    | Archivo clave |
|--------------------|-------------------------------------------------|---------------|
| 1. Fundamentos     | Filosofia, 4 capas, carpetas, flujo de datos    | 01-04         |
| 2. Implementacion  | CRUD completo: Domain, Data, Presentation+UI    | 05-05c        |
| 3. Infraestructura | DI con GetIt, templates, decisiones             | 06-08         |
| 4. Madurez         | Migracion, SOLID, anti-patrones, cuando NO usar | 09-12         |

## 1 Filosofia y analogias (2 min)

ROBERT C. MARTIN · UNCLE BOB

Clean Architecture = separar responsabilidades en capas concetricas. El nucleo (Domain) no depende de nada externo.

### La analogia del restaurante

| Restaurante  | Codigo             | Capa         |
|--------------|--------------------|--------------|
| Receta       | Entity, UseCase    | DOMAIN       |
| Cocina       | Model, DataSource  | DATA         |
| Mesero       | Cubit, State       | PRESENTATION |
| Mesa/Cliente | Widget, Page       | UI           |
| Almacen      | API, Base de datos | EXTERNAL     |

### La analogia del edificio de 4 pisos

|                      |                                   |
|----------------------|-----------------------------------|
| Piso 4 (arriba): UI  | - El cliente ve y toca            |
| Piso 3: Presentation | - El mesero orquesta              |
| Piso 2: Domain       | - El chef cocina (nucleo puro)    |
| Piso 1 (abajo): Data | - La cocina accede a ingredientes |

4 beneficios: Independencia de frameworks · Testable al 100% · Escalable y mantenible · Organizacion clara

### El problema: codigo espagueti

```
Widget build(BuildContext context) {
  final response = await http.get(Uri.parse('https://api...')); // HTTP en UI!
  final data = jsonDecode(response.body); // Parseo en UI!
  if (data['status'] == 200) { ... } // Logica en UI!
}
```

Todo mezclado: HTTP + parseo + logica + UI en un solo lugar. Clean Architecture resuelve esto.

## 2 Las 4 capas en detalle (3 min)

| Capa         | Contiene                              | Depende de           | Analogia                |
|--------------|---------------------------------------|----------------------|-------------------------|
| Domain       | Entity, Repository Interface, UseCase | Solo Dart puro       | Reglas del ajedrez      |
| Data         | Model, DataSource, RepositoryImpl     | Domain (interfaces)  | Tablero fisico          |
| Presentation | Cubit/Bloc, State                     | Domain (UseCases)    | Visor movil del tablero |
| UI           | Widgets, Pages                        | Presentation (Cubit) | Pantalla del telefono   |

**Test de capa:** Si el codigo puede vivir en un proyecto Dart sin Flutter → es Domain. Si no → es otra capa.

## 3 principios fundamentales

- **Capas:** Cada archivo pertenece a UNA sola capa
- **Regla de dependencia:** Las flechas van de afuera hacia adentro (UI → Presentation → Domain ← Data)
- **Abstracciones:** Domain usa interfaces/abstract classes; Data las implementa

## Flujo de llamada

```
UI(UserWidget) → Cubit.fetchData() → UseCase.call()
  → Repository.getData() [interface en Domain]
  → RepositoryImpl [en Data, implementa la interface]
  → DataSource.fetch() [API/DB]
  → Modelo → Entity → Either<Failure, Entity> → Cubit → UI
```

## 3 Estructura de carpetas universal (2 min)

```
lib/
  core/
    common/      use_case.dart    # Compartido entre features
    di/          injection.dart   # UseCase base abstracto
    error/       failures.dart    # GetIt service locator
    network/     network_info.dart # Clases de error
    routing/     app_router.dart
    theme/       app_theme.dart
  features/
    {feature}/
      data/
        datasources/ # Remote + Local data sources
        models/      # Modelos (DTO, JSON, Isar)
        repositories/ # RepositoryImpl
      domain/
        entities/    # Entidades puras
        repositories/ # Interfaces abstractas
        usecases/    # Casos de uso
      presentation/
        cubit/       # Cubit + States
        pages/       # Pantallas
        widgets/     # Widgets reutilizables
```

## Reglas de organizacion

| Hacer (Dos)                      | No hacer (Donts)                   |
|----------------------------------|------------------------------------|
| Una carpeta por feature          | Agrupar todos los data juntos      |
| Features independientes entre si | Agrupar todas las entities juntas  |
| Core NO depende de features      | Agrupar todas las pantallas juntas |
| Cada layer en su carpeta         | Poner logica de negocio en widgets |

## 4 Flujo de datos y manejo de errores (3 min)

Cada operacion CRUD sigue una **forma de U**: baja por las capas y sube de vuelta.

```
USUARIO toca boton
  → Widget: context.read<UserCubit>().fetchUsers()
  → Cubit: emit UserLoading(); result = await useCase();
  → UseCase: valida reglas, llama repository
  → RepositoryImpl: decide remote vs local
  → DataSource: ejecuta HTTP/DB query
  → Modelo: UserModel.fromJson() → toEntity() → UserEntity
  → Either<Failure, UserEntity> sube de vuelta
  → Cubit: emit UsersLoaded(users) o UserError(message)
  → UI: BlocBuilder reconstruye el widget
```

### Manejo de errores: Exception vs Failure

| Concepto  | Capa   | Que es                              | Ejemplo                                     |
|-----------|--------|-------------------------------------|---------------------------------------------|
| Exception | DATA   | Error tecnico (no llega al usuario) | ServerException, CacheException             |
| Failure   | DOMAIN | Error amigable (llega al usuario)   | ServerFailure, CacheFailure, NetworkFailure |

### Cadena de errores

```
DataSource lanza Exception
  → RepositoryImpl catch(Exception) → convierte a Failure
  → 返回 Either<Failure, T> (Left)
  → UseCase recibe Failure
  → Cubit emite UserError(message)
  → UI muestra SnackBar amigable
```

**Nunca dejes que una excepcion tecnica (HTTP 500, timeout, SQL) llegue a la UI. Siempre se convierte en Failure primero.**

## 5 Domain Layer — El nucleo puro (3 min)

Contiene 3 componentes: **Entity**, **Repository Interface**, **UseCase**. Sin dependencias externas.

### Entity

```
class User extends Equatable {
    final String id;
    final String name;
    final String email;
    final bool isActive;
    final DateTime createdAt;

    const User({required this.id, ...});

    User copyWith({String? name, ...}) => User(id: id, name: name ?? this.name, ...);

    bool get isNew => DateTime.now().difference(createdAt).inDays < 7;

    @override
    List<Object?> get props => [id, name, email, isActive, createdAt];
}
```

### Repository Interface

```
abstract class UserRepository {
    Future<Either<Failure, List<User>>> getUsers();
    Future<Either<Failure, User>> getUser(String id);
    Future<Either<Failure, User>> createUser(User user);
    Future<Either<Failure, void>> deleteUser(String id);
}
```

### UseCase

```
class GetUsers extends UseCase<List<User>, NoParams> {
    final UserRepository repository;
    GetUsers(this.repository);

    @override
    Future<Either<Failure, List<User>>> call(NoParams params) async {
        return await repository.getUsers();
    }
}
```

### Future<Either> vs TaskEither

| Aspecto     | Future<Either>     | TaskEither                 |
|-------------|--------------------|----------------------------|
| Complejidad | Baja (familiar)    | Alta (funcional)           |
| Composicion | Limitada           | Encadenable con .andThen() |
| Ideal para  | Equipos junior/mid | Equipos avanzados          |

El modulo usa Future<Either> por accesibilidad.

## 6 Data Layer — La implementacion (4 min)

Implementa los contratos de Domain. Contiene: **Model**, **DataSource**, **RepositoryImpl**.

### Model (con Isar para DB local)

```
class UserModel extends User {
    const UserModel({required super.id, ...});

    factory UserModel.fromEntity(User user) => UserModel(id: user.id, ...);
    User toEntity() => User(id: id, name: name, ...);

    // Isar: @collection, @Index(), toEntity(), fromEntity()
    // REST: fromJson(), toJson(), fromEntity(), toEntity()
}
```

### DataSource — Dos opciones

| Aspecto    | Dio (REST API)                 | SupabaseClient                       |
|------------|--------------------------------|--------------------------------------|
| Ideal para | APIs REST externas             | Backend Supabase (BaaS)              |
| Error      | DioException → ServerException | PostgrestException → ServerException |
| Query      | response.data                  | from().select().order()              |
| Auth       | Headers manuales               | SupabaseClient.auth                  |

### RepositoryImpl — Logica Online/Offline

```
class UserRepositoryImpl implements UserRepository {
    final UserRemoteDataSource remoteDataSource;
    final UserLocalDataSource localDataSource;
    final NetworkInfo networkInfo;

    @override
    Future<Either<Failure, List<User>>> getUsers() async {
        if (await networkInfo.isConnected) {
            try {
                final models = await remoteDataSource.getUsers();
                await localDataSource.cacheUsers(models); // Guardar en cache
                return Right(models.map((m) => m.toEntity()).toList());
            } on ServerException {
                return Left(ServerFailure());
            }
        } else {
            try {
                final cached = await localDataSource.getCachedUsers();
                return Right(cached.map((m) => m.toEntity()).toList());
            } on CacheException {
                return Left(CacheFailure());
            }
        }
    }
}
```

### UserSession — Cuando inyectarla

| Repositorio     | Necesita UserSession? | Por que                     |
|-----------------|-----------------------|-----------------------------|
| UserRepository  | No                    | Gestiona todos los usuarios |
| OrderRepository | Si                    | Filtra por usuario actual   |
| AuthRepository  | Si                    | Maneja login/logout         |

## 7 Presentation + UI Layer — El estado y la vista (3 min)

### States

```
abstract class UserState extends Equatable {}
class UserInitial extends UserState { ... }
class UserLoading extends UserState { ... }
class UsersLoaded extends UserState {
    final List<User> users;
    const UsersLoaded(this.users);
}
class UserError extends UserState {
    final String message;
    const UserError(this.message);
}
```

### Cubit

```
class UserCubit extends Cubit<UserState> {
    final GetUsers getUsers;
    final GetUser getUser;
    final CreateUser createUser;
    final DeleteUser deleteUser;

    UserCubit({required this.getUsers, ...}) : super(UserInitial());

    Future<void> loadUsers() async {
        emit(UserLoading());
        final result = await getUsers(NoParams());
        result.fold(
            (failure) => emit(UserError(_mapFailure(failure))),
            (users) => emit(UsersLoaded(users)),
        );
    }
}
```

### UI con flutter\_bloc

```
BlocProvider(
  create: (context) => sl<UserCubit>()..loadUsers(),
  child: BlocBuilder<UserCubit, UserState>(
    builder: (context, state) {
      if (state is UserLoading) return CircularProgressIndicator();
      if (state is UserError) return Text(state.message);
      if (state is UsersLoaded) {
        return ListView.builder(
          itemCount: state.users.length,
          itemBuilder: (ctx, i) => ListTile(user: state.users[i]),
        );
      }
      return SizedBox();
    },
  ),
)
```

**Regla: UI solo muestra datos y captura eventos. Nunca logica de negocio, ni llamadas HTTP, ni parseo de JSON.**

## 8 Inyeccion de Dependencias con GetIt (2 min)

GetIt es un **Service Locator**: registra instancias y las entrega cuando se necesitan.

### 3 metodos de registro

| Metodo                | CUANDO se crea          | Ideal para                              |
|-----------------------|-------------------------|-----------------------------------------|
| registerLazySingleton | Primera vez que se pide | Repositorios, DataSources (recomendado) |
| registerSingleton     | Al registrarlo          | Config, servicios globales              |
| registerFactory       | Cada vez que se pide    | Cubits (nueva instancia por uso)        |

### Estilo Manual (recomendado)

```
final sl = GetIt.instance;

Future<void> init() async {
  // — Externos —
  sl.registerLazySingleton(() => SupabaseClient());
  sl.registerLazySingleton(() => InternetConnection());

  // — Core —
  sl.registerLazySingleton(() => NetworkInfoImpl(sl()));

  // — DataSources —
  sl.registerLazySingleton<UserRemoteDataSource>(
    () => UserRemoteDataSourceImpl(sl()),
  );
  sl.registerLazySingleton<UserLocalDataSource>(
    () => UserLocalDataSourceImpl(),
  );

  // — Repositories —
  sl.registerLazySingleton<UserRepository>(
    () => UserRepositoryImpl(
      remoteDataSource: sl(),
      localDataSource: sl(),
      networkInfo: sl(),
    ),
  );

  // — UseCases —
  sl.registerLazySingleton(() => GetUsers(sl()));
  sl.registerLazySingleton(() => CreateUser(sl()));

  // — Presentation —
  sl.registerFactory(() => UserCubit(getUsers: sl(), createUser: sl()));
}
```

### Estilo Injectable (proyectos grandes)

Usa anotaciones (@lazySingleton, @injectable) + build\_runner. Requiere: injectable, injectable\_generator.

**Regla: Empieza manual. Cambia a injectable cuando el service\_locator.dart tenga +200 lineas.**

## 9 SOLID en Dart/Flutter + Anti-patrones (3 min)

### Los 5 principios SOLID

| # | Principio             | Regla                                       | Ejemplo en Dart                                      |
|---|-----------------------|---------------------------------------------|------------------------------------------------------|
| S | Single Responsibility | Una clase, una razon para cambiar           | CreateUserUseCase (no UserManager)                   |
| O | Open/Closed           | Abierto a extension, cerrado a modificacion | Abstract class + implementaciones                    |
| L | Liskov Substitution   | Subtipos deben ser sustituibles             | Ambos implementan Shape (no hereda Rectangle→Square) |
| I | Interface Segregation | Interfaces pequenas y especificas           | UserReader + UserWriter (no UserRepository gigante)  |
| D | Dependency Inversion  | Depender de abstracciones, no concreciones  | Interface + inyeccion + GetIt                        |

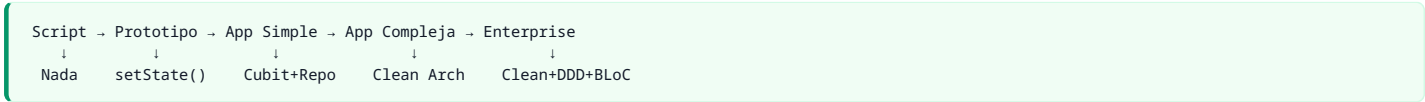
### Los 8 anti-patrones

| # | Anti-patron                 | Problema                                | Solucion                                    |
|---|-----------------------------|-----------------------------------------|---------------------------------------------|
| 1 | God Class                   | Una clase hace todo                     | Separar por responsabilidad                 |
| 2 | Dependencia invertida (mal) | Domain depende de Data/Presentation     | Flechas solo hacia Domain                   |
| 3 | Repository de papel         | Repository sin logica, solo passthrough | Agregar logica o fusionar con UseCase       |
| 4 | Estado monolitico           | Un solo State con 10+ campos            | Dividir por dominio o usar multiples Cubits |
| 5 | UseCase castrado            | UseCase solo delega sin logica          | Agregar regla de negocio o fusionar         |
| 6 | Map hell                    | Map<String, dynamic> sin tipos          | Siempre usar Modelos tipados                |
| 7 | Herencia infinita           | 5+ niveles de abstracciones             | Composicion sobre herencia                  |
| 8 | Callback hell               | Callbacks anidados profundos            | async/await + manejo de errores             |

**Test rapido:** Si tu archivo tiene +300 lineas, si un UseCase solo delega, o si un State tiene +10 campos → probablemente tienes un anti-patron.



El spectrum de complejidad



Cuando SI usar

- ✓10+ features
- ✓3+ desarrolladores
- ✓Vida util de 2+ anios
- ✓ multiples fuentes de datos
- ✓Testing exhaustivo
- ✓Dominio complejo

Cuando NO usar

| Caso                 | Alternativa               |
|----------------------|---------------------------|
| Prototipo / MVP      | setState() + http         |
| App de una pantalla  | Cubit simple              |
| Script / herramienta | Sin arquitectura          |
| Hackathon            | Lo que funcione           |
| UI-only app          | Widgets + setState        |
| POC interno          | Cubit + Repository simple |

Senales de over-engineering

| Senal                                  | Accion                              |
|----------------------------------------|-------------------------------------|
| Mas archivos que features              | Eliminar abstracciones innecesarias |
| UseCases que solo delegan              | Fusionar con Repository             |
| Interfaces con una sola implementacion | Eliminar la interfaz                |
| 3+ niveles de abstraccion              | Aplanar la jerarquia                |
| Tardas 30+ min en agregar una feature  | Simplificar la arquitectura         |

Regla practica: 3+ features = Clean Architecture. Menos = Cubit + Repository simple. Desechable = lo que funcione.

## A Anexo · Índice del modulo (para profundizar)

| Necesitas                                       | Archivo                                 |
|-------------------------------------------------|-----------------------------------------|
| Filosofia y analogias de Clean Architecture     | 01-introduccion-y-filosofia.md          |
| Las 4 capas explicadas en detalle               | 02-las-4-capas.md                       |
| Estructura de carpetas universal                | 03-estructura-de-carpetas.md            |
| Flujo de datos completo con diagrama Mermaid    | 04-flujo-de-datos.md                    |
| Introduccion al CRUD de ejemplo (Users)         | 05-implementacion-crud-intro.md         |
| Domain Layer: Entity, Repository, UseCase       | 05a-domain-layer.md                     |
| Data Layer: Model, DataSource, RepositoryImpl   | 05b-data-layer.md                       |
| Presentation + UI: States, Cubit, Widgets       | 05c-presentation-ui-layer.md            |
| Inyeccion de Dependencias con GetIt             | 06-inyeccion-de-dependencias.md         |
| Templates universales para cada capa            | 07-templates-universales.md             |
| Decisiones de arquitectura (Cubit vs BLoC, FAQ) | 08-decisiones-de-arquitectura.md        |
| Migracion gradual desde codigo espagueti        | 09-migracion-codigo-espagueti.md        |
| SOLID explicado con Dart/Flutter                | 10-solid-explicite.md                   |
| 8 anti-patrones en Clean Architecture           | 11-anti-patrones-clean-architecture.md  |
| Cuando NO usar Clean Architecture               | 12-cuando-no-usar-clean-architecture.md |

## C Conclusion

Clean Architecture es una **herramienta, no un dogma**. Usala cuando el proyecto lo necesite. La clave es:

- ✓Domain puro: sin dependencias externas
- ✓Flechas hacia adentro: Domain nunca sabe de UI ni Data
- ✓Interfaces en Domain, implementaciones en Data
- ✓Errors: Exception (DATA) → Failure (DOMAIN) → UI amigable
- ✓GetIt para DI: manual para empezar, injectable para escalar
- ✓SOLID como guia, no como religion

Tiempo de lectura del resumen: ~25 min · Clean Architecture es un camino, no un destino.